

CSC7062 Report

Project link: <https://gitlab.eecs.qub.ac.uk/40343604/hedclass>

User Credentials:

- **Registry Administrator:** Username: registry_admin | Password: password123
- **Classification Officer:** Username: John_Smith | Password: welcome123

If there are any issues or problems accessing the above, please do not hesitate to contact me.

The next section of the report shall provide a summary of the implemented functions. This report shall look at each section of the functions individually and will provide examples of how such sections or sub-sections were achieved.

A. Authentication:

- **[A1] Login and Logout (Must Have):** This was implemented successfully. Authentication is handled with a secure `/login` POST route. The system can extract the user credentials and queries the MySQL database. If the credentials are valid, the user is then authenticated and can access the dashboard suitable for their role. The `/logout` GET route terminates access through the `req.session.destroy()`, thus wiping the server-side memory and redirecting the user back to the login view.
- **[A2] Maintain State (Must Have):** This was implemented successfully. The application utilises the express-session middleware. Upon successful authentication user identifiers such as `isLoggedIn`, `username`, `role`, `user_id` are persistently stored within the session object. Subsequent route requests rely heavily on the `req.session.isLoggedIn` Boolean to maintain continuous and secure access across the platform without repeated login prompts.
- **[A3] Role-Based Access Control (Must Have):** This was implemented successfully. The database assigns each user a definitive system role (`registry_admin` or `classification_officer`). The base route (`/`) evaluates the `req.session.role` variable and automatically redirects authenticated users to their respective and restricted dashboards. There is strict logical separation between the primary and secondary actors.

B. Secondary Actor Role:

- **[B1] Create and Manage Classification Officers (Must Have):** This was implemented successfully via the `/add-officer` and `/delete-officer` POST routes. Upon creating an officer, the system executes a successful `INSERT INTO users` statement. The system then captures the newly generated `insertId` to create a linked record in the `officer_assignments` table. Upon deleting an officer, the system deletes their assignments before deleting the user record. This was designed in place to ensure integrity.
- **[B2] Assign Classification Officer to Specific Degree/Programme (Must Have):** This was implemented successfully. The frontend EJS template populates dynamically a `<select>` dropdown with available programmes. The administrator is then able to select a valid `programme_id`.
- **[B3] Create Degree/Programme with Base Classification Rules (Should Have):** This was implemented successfully. Administrators manage programmes via the `/add-programme`, `/update-programme`, and `/delete-programme` routes. The system enforces strict server-side business logic validation: before any `INSERT` or `UPDATE` SQL queries are executed, the system calculates the combined Year 2 and Year 3 weightings. If the total does not equal exactly `1.0`, the system rejects the database write.
- **[B4] Assign Classification Officer to More than One Degree (Should Have):** This was implemented successfully. The junction table (`officer_assignments`) supports a many-many-relationship between users and programmes. The `/assign-officer` POST route inserts additional rows into this table, thus allowing a single classification officer the ability to be responsible for multiple distinct students without duplicating the user accounts.

C. Primary Actor:

- **[C1] Classification Dashboard (Must Have):** This was implemented successfully. Access to the Classification Officer dashboard is strictly protected by session validation (`req.session.role !== 'classification_officer'`). The initial view queries the `officer_assignments` junction table using an `INNER JOIN` to ensure the officer only sees degree programmes they are specifically authorised to manage.
- **[C2] Cohort (candidate) data management (Must Have):** Classification officers possess full CRUD actions over the students assigned to them. Officers can add and

delete students safely. The delete operation maintains database integrity by executing a `DELETE FROM module_results` prior to removing the student record. Overall, this was implemented successfully.

- **[C3] Automate Classification Rules (Must Have):** This was implemented successfully. The backend calculates classifications within the `/review/:student_id` route. The algorithm iterates through the module results utilising the `Math.min(module.mark, 40)` to strictly cap the resit marks. Additionally, it aggregates the total credits to maintain the 120-credit-per-year progression rule. Finally, to conclude, it calculates the `finalScore` utilising the dynamically parsed `y2_weighting` and `y3_weighting` multipliers before evaluating the result against the tier boundaries such as `finalScore >= 70`.
- **[C4] Review and manage individual student records (Must Have):** This was successfully implemented. The `review.ejs` view aggregates the student's full module history. Classification officers can utilise the `/add-grade` POST route to include or update new results.
- **[C5] Decision Rationale Log (Should Have):** This was implemented successfully. Classification officers are able to bypass the automated or calculated classification with the `/override/:student_id` POST route. The system requires both the `override_classification` and a text-based `rationale`. These are later updated in the `students` table and are rendered on the EJS template.
- **[C6] Classification Distribution Summary (Should Have):** This was implemented successfully. When the classification officer selects a cohort or programme the dashboard provides a chart of the students currently enrolled in that programme with their respective grades.
- **[C7] Export option for board of examiners (Could Have):** This was not implemented.
- **[C8] Institute Level Classification Rules (Could Have):** This was implemented successfully. The algorithm automatically validates progression compliance by ensuring a candidate has met the required thresholds. Furthermore, it strictly enforces the institutional resit policy by identifying flagged modules and capping them at the respective module mark.
- **[C9] Defined Per Programme Rules (Could Have):** This was implemented successfully. Rather than relying on a static and hardcoded calculation model for the

entire institute, this architecture manages classification weightings dynamically at the individual programme level. The registry administrator can adjust the year two and year three weightings. This allows for vastly different degrees to be calculated securely and accurately through the same backend endpoint.

D. Non-functional

- **[D1] Accuracy (Must Have):** This was successfully implemented. An example would be, to provide JavaScript floating-point errors during weighting calculations, the system uses `parseFloat()` and rounds to two decimal places. This system guarantees deterministic and consistent rule execution.
- **[D2] Scalability (Must Have):** This was successfully implemented. This application was built on Node.js and can handle multiple concurrent requests. Additionally, the underlying MySQL database adopts relational mapping to prevent any data duplication. This ensures that the performance remains stable and is able to perform as intended as the user base expands.
- **[D3] Base security measures (Must Have):** This was successfully implemented. Unauthorised access is prevented via strict server-side session validation (`req.session.role`) on every restricted route. Additionally, queries often are using `?` as placeholders across all database interactions.
- **[D4] Design (Must Have):** This was successfully implemented. The application features a modern and responsive user interface. The user interface was constructed via Bootstrap. This provides a mobile-first, fluid grid system, with accessible interactive components.
- **[D5] Versioning Control (Must Have):** This was successfully implemented. Source code management and version tracking were maintained using Git. Frequent atomic commits were made to ensure a traceable development history. This will be accessed in the `gitlog.txt` file.
- **[D6] Auditability (Could Have):** This was successfully implemented. The system maintains full traceability of grading anomalies. The database strictly requires and permanently logs a `decision_rationale` string tied to the officer's `user_id`, providing a transparent audit trail.

E. System Architecture

- **[E1] Modular Node/Express Architecture (Should Have):** This was successfully implemented. The backend utilises the `express.Router()` class to enforce a strict separation of concerns. I decided that the `app.js` entry point would delegate the server initialisation and middleware configuration. Database connection is additionally made accessible via a standalone `db.js` file.
- **[E2] Standalone REST API (Should Have):** This was successfully implemented. The architecture of this application incorporated a decoupled RESTful API endpoint to interface with the frontend. The `GET /officer/api/students/:programme_id` route acts as a microservice, querying the MySQL database using the `req.params` identifier and returning the dataset via `res.json()`. This allows the client-side JavaScript to asynchronously fetch and render the cohort data and Chart.js graphics without interrupting the user experience with full page reloads.